RRCI

Table of contents

Introduction

Introduction to Rolling Roof Computer Interface (RRCI)

RRCI is designed for to help astronomers automate their roll-off-roof observatories for remote access.

RRCI makes no warranty as to the operation or automation of the observatory equipment; it simply provides components for 'Do It Yourself' automation projects.

Caution! Automation of any rolling roof observatory requires special attention with regard to the location of individuals in the observatory, as well as the position of the telescope/mount, prior to moving the roof to ensure no one or nothing obstructs the roofs path. Serious injury to individuals and/or damage to telescope, mount or the observatory may occur if this advice is not followed. We recommend that all users of RRCI implement safety policies and procedures regarding the operation of their rolling roof. In addition we strongly recommend installation of safety sensors that will prevent or stop movement of the roof should individuals or equipment move into an unsafe position before or during roof movement. For remote/automated operation of rolling roofs it is recommended that the "*Allow Remote Apps to Open/Close the Roof*"" check-box in the RRCI Live software be unchecked if the roof does not safely clear the scopes/mounts. Misuse or careless operation can cause serious damage to your equipment or individuals. The author of the software and the distributor of the equipment (1) make no warranty or representation to the user as to the operation of this System and (2) is NOT responsible for any injury or damage that may occur from careless implementation, operation or misuse. **Opening or Closing a roll-off roof is not a trivial task and the operator needs to take all necessary precautions before starting the roof motor. Damage to equipment or persons can occur when the roof is in motion so all necessary precautions should be taken before executing such a command.**

Release Notes

Release Notes

See Github

License Agreement

Credits and Copyright

Credits and Copyright

Trademarks

Trademarks

RRCI is a trademark of CCDASTRO, Inc. Windows are registered trademarks of Microsoft Corporation. Other product names mentioned in this publication may be trademarks of other companies.

Support

I will try to help via email:

System Requirements

System Requirements

RRCI requires Windows 10 (or higher, x64).

The following hardware is required:

- RRCI System Arduino hardware/firmware sketch

Installation

Installation

RRCI is available via Internet download.

Important: RRCI require the installation of the ASCOM Platform, available at http://ascom-standards.org.

Internet Installation

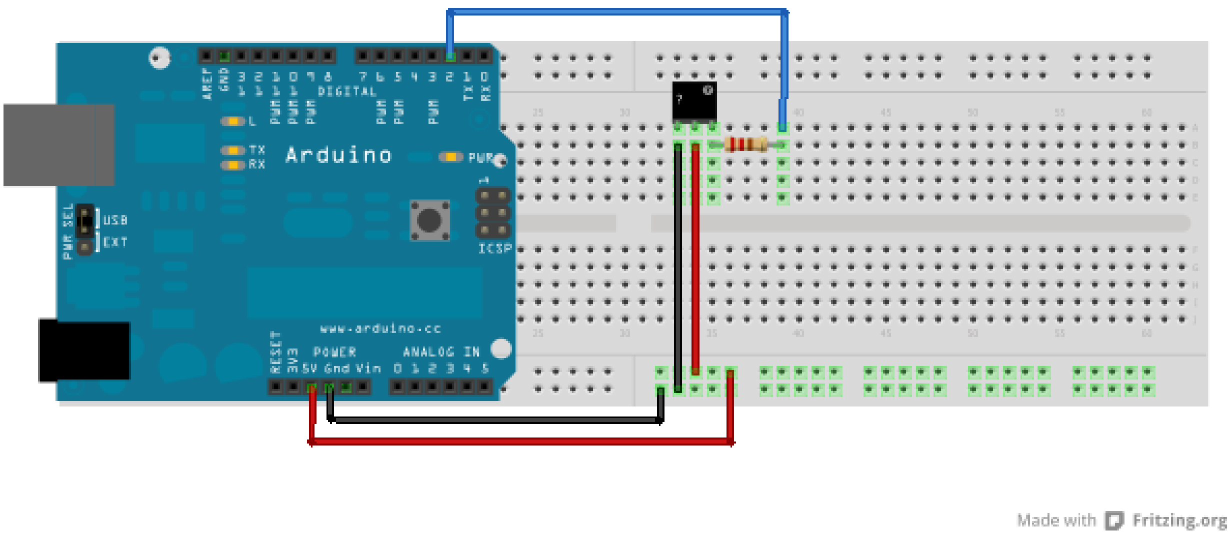
1. Download the latest RRCI ASCOM driver and software here .
2. You will be prompted to either 'Run' the installer or you can save it to a folder; double clicked the saved file with start the installer.
3. Follow the onscreen instructions to complete the installation.

4. Once the software has successfully installed you must enter the setup areas to input the proper setting before attempting to connect your hardware. To properly setup RRCILive open the programs from Windows/Start/RRCI.

Scope Position Sensor

Any normally open (unsafe) device that will be closed (safe) when mount in a safe position for roof movement.

Roof Movement Hall Sensor Wiring



Basic RRCI Operation

Basic Operation

In the following discussion, it is assumed that the reader is generally familiar with ASCOM client programs (NINA, APP, SGP, Prism, Etc.) that run under the Windows operating system.

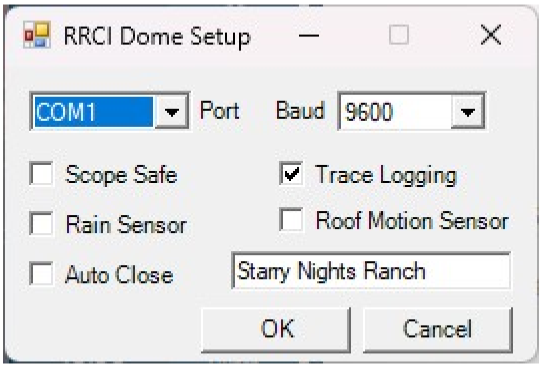
Running RRCI

Basic Operation

RRCI is an ASCOM Dome V2 driver accessible by any ASCOM client via the Dome Chooser.

Setup

Setup Tab:



RRCI Communications:

Port: input the Com Port - this should be automatically populated

Baud - generally 9600

Features:

Scope Safety - Checking this box will require a normally open safety to determine a safe telescope/mount position before moving.

Rain Sensor - Checking this box will enable rain sensor.

Auto Close - this will auto close if rain and scope safe.

Trace Logging - log activities.

Roof Motion Sensor - check if hall sensor installed.

Text Box - Name, location or whatever you want.....

Roof Movement Sensor

Hall Sensor to Report Roof Movement

Better than a fixed timeout

A fixed (currently used in Arduino firmware sketch timeout only answers:

“Did the roof finish within 120 seconds?”

A Hall sensor answers:

“Is the roof actually moving right now?”

That is a much more meaningful check.

How the current firmware uses it

When the Motion Sensor is enabled via the checkbox in setup:

1. The Hall sensor produces pulses as the roof moves.
2. The firmware records the time of the last pulse.
3. If no pulse is detected for 3 seconds (modify in sketch as needed) while opening or closing:
    - The controller enters `ERROR`.
4. The ASCOM driver sees `STATE:ERROR`.

So the pulse count is used as a real-time motion watchdog.

Additional uses for pulse counts

You could also use pulse counts to:

- Estimate travel distance
- Detect gradual mechanical wear
- Measure opening and closing consistency
- Set expected pulse ranges

For example:

- Fully open typically takes ~320 pulses.
- If the roof stops after 47 pulses, something is wrong. A person could determine how far it travel in that period.

Pulse count vs pulse timing

The most important information is usually:

- Are pulses arriving?
- How long since the last pulse?

The total count is useful for diagnostics and future enhancements.

Future enhancement

Compare pulse count to expected travel:

```
If fewer than 250 pulses occur before the open limit switch activates,
flag a possible mechanical issue.
```

That is optional but can be helpful.

Detect Actual Position of Roof

Hall sensor pulse count is highly valuable internally because it confirms that the roof is genuinely moving and allows the controller to detect stalls or mechanical failures much more reliably than timeouts alone.

Instead of placing magnet or sensor on a spinning part, have magnets on the roof at intervals that produce a pulse when that part of the roof passes

How this works

Instead of mounting a magnet on a rotating part of the motor, mount multiple magnets along the roof (or along the rail), and place a stationary Hall sensor at a fixed point.

As the roof moves past the sensor, each magnet produces one pulse.

Calibrate Hall Sensor Pulses

When the **Calibrate** button is clicked, the system performs a fully automated roof-travel learning cycle.

Here is the complete sequence.

---

1. User Clicks “Calibrate”

Inside `StatusForm`:

```
dome.Action("Calibrate",  "");
```

is sent to the active ASCOM driver.

This avoids:

- creating another driver instance,
- duplicate COM connections,
- serial conflicts.

---

2. Driver Receives ASCOM Action

Inside:

```
Action()
```

the driver detects:

```
Calibrate
```

and calls:

```
CalibrateOpenPulses()
```

---

3. Driver Resets Arduino Pulse Counter

Driver sends:

```
resetpulse#
```

to Arduino.

Firmware executes:

```
motionPulseCount = 0;
```

Now:

- roof closed position = 0 pulses.

---

4. Driver Starts Roof Opening

Driver calls:

```
OpenShutter()
```

which sends:

```
open#
```

to Arduino.

Firmware:

- enters `OPENING`,
- starts watchdog timing,
- begins pulse counting.

---

5. Hall Pulses Begin Counting

As roof moves:

- hall sensor pulses are detected,
- `motionPulseCount` increments continuously.

Example:

```
1
2
3
...
4821
```

---

6. Driver Polls Roof State

During calibration loop:

```
ShutterStatus
```

is repeatedly checked.

Driver waits for:

```
shutterOpen
```

which occurs when:

- open limit switch activates.

---

7. Final Pulse Count Captured

Driver reads:

```
RoofTelemetry.CurrentPulseCount
```

Example:

```
4821
```

---

8. Safety Margin Added

Driver applies tolerance:

```
4821 × 1.02
```

Example:

```
4917
```

This helps tolerate:

- temperature expansion,
- belt wear,
- small pulse variance.

---

9. Calibration Saved

Driver stores:

```
OpenPulseCount
```

to ASCOM profile settings.

Now calibration persists across:

- reboot,
- reconnect,
- NINA restart.

---

10. Future Roof Movement Uses Calibration

Now all telemetry becomes accurate:

- percent open,
- progress bar,
- overshoot protection,
- movement monitoring.

Example:

```
2450 pulses = 50% open
```

---

11. Live UI Updates Automatically

During calibration the status window shows:

- Opening
- live pulse count
- live percentage
- progress bar movement

in real time.

RESULT

The system now:

- automatically learns roof travel distance,
- stores calibration permanently,

- and enables accurate telemetry-driven operation without manual pulse measurement.

Scripting

Scripting

RRCI provides an ActiveX Automation interface for scripting and externally controlling the Dome via the conventions outlined in the ASCOM Platform documents. To access these documents refere to the developer area in your ASCOM folders contained in your Windows Start menu.

WeatherWatcher's is also an ActiveX Automation interface for scripting and externally accessing the Watcher.

ActiveX can be accessed from just about any standard scripting or programming language, including VBScript, JScript, Java, Perl, Visual Basic, Visual C++, etc. Using ActiveX, you can even control RRCI from Excel spreadsheet macros.

RRCI is compliant with ASCOM scripting. This allows it to operate with a wide variety of astronomical software products such as planetarium programs, telescope control software, and dome control systems. Sample scripts are available for downloading from the ASCOM web pagehttp://ascom-standards.org.

Scripting Languages

The most commonly-used scripting languages on the Windows platform are VBScript and JScript.

VBScript looks like a simplified version of Visual Basic (thus the name), but you *do not* require Visual Basic to create or run scripts. VBScript code is run by the Windows Scripting Host, which is included in Windows.

Similarly, JScript looks like Java code. JScript is run by the very same Windows Scripting Host that runs VBScript. Deciding which version to use is simply a matter of user preference.

VBScript and JScript allow you to write simple scripts using only the **Notepad** text editor, and run by double-clicking them in the Explorer.

Full documentation on the scripting languages is also available for download; please see Windows Scripting Reference,

Objects:

```
RRCI.Dome  and  WeatherWatcher.Watcher
```


Sample Scripts

Sample Scripts

In VBScript, you can access the RRCI dome as follows:

```
Dim RRCI ' "The" Dome object
Set RRCI = CreateObject("RRCI.Dome")
RRCI.Connected = True

if Not RRCI.Connected Then
    wscript.echo "Failed to connect to the RRCI."
   Quit
End If
```

```
wscript.echo   RRCI.Description

RRCI.Connected  =  False
```

If you want to try this, just copy the above code into a text file named "test.vbs". Double-click on the file to run it.

In Visual Basic you can access the RRCI Dome object the same way. However, another method is available which enables the Intellisense feature of Visual Basic: click References… under the Project menu. Scroll down to RRCI ASCOM Dome Controller and turn on the appropriate check box and click OK. You can then create a RRCI dome object:

```
Dim RRCI As New RRCI Dome'RRCI Dome object
RRCI.Connected = True 'Link

If Not RRCI.Connected Then
      MsgBox "Failed to start RRCI."
      End
End If

MsgBox  RRCI.Description

RRCI.Connected  =  False
```


Windows Scripting Reference

Windows Scripting Reference

Scripting Home Page

A variety of information is available on Microsoft's Scripting Home Page at http://msdn.microsoft.com/en-us/library/ms950396.aspx. Language definitions for VBScript and JScript are included on this page.

ASCOM Initiative

A master site for astronomical scripting, which includes sample scripts using RRCI and other applications, is available on the ASCOM web page http://ascom-standards.org.

ASCOM Properties and Methods

ASCOM Properties and Methods

Please refer to the ASCOM web site http://ascom-standards.org for a description of the Properties and Methods used in the ASCOM-standard interfaces.

File Locations

All data files needed and/or created by m1Live are saved in the following locations:

Vista/Win7/ Win 8 = ...\ProgramData\RRCI folder
XP = ..\Documents and Settings\All Users.WINDOWS\Application Data\RRCI